Digital Asset Exchange Architecture: From High-Frequency Trading to Institutional-Grade Order Matching

Zach Kelling

Hanzo AI

2019

Abstract

We describe the evolution of digital asset exchange architecture at Hanzo Industries, from the Lightspeed DEX concept (2017) through the centralized exchange implementation (August 2019, 636 commits). The centralized exchange implements a price-time priority order matching engine, multi-venue order routing, and a WebSocket-based market data feed. We formalize the order matching algorithm, describe the risk management layer, and present the system architecture that subsequently evolved into the Lux Exchange execution stack. We reference the upstream Uniswap automated market maker (2017) as the foundational AMM design that informed the decentralized component.

1 Introduction

Digital asset exchanges must reconcile two competing requirements: the throughput and determinism of traditional financial matching engines, and the transparency and self-custody guarantees of decentralized protocols. Between 2017 and 2019, Hanzo developed exchange infrastructure spanning both paradigms.

The Lightspeed DEX concept (2017) explored high-frequency on-chain trading, pushing the limits of what was achievable on Ethereum L1 at the time. The centralized exchange (CEX) implementation, begun in August 2019 with 636 commits, provided the order matching engine, account system, and market data infrastructure required for institutional-grade trading.

2 Lightspeed DEX (2017)

2.1 Concept

The Lightspeed DEX was an early attempt at on-chain order book trading. The design placed limit orders on-chain as contract state and matched them in a priority queue maintained by the contract. The upstream Uniswap constant product AMM [1], published the same year, took the alternative approach of replacing the order book with a bonding curve.

2.2 Limitations

The on-chain order book approach was ultimately constrained by Ethereum L1 gas costs and block times. Placing, canceling, and matching orders each required a transaction, making the system impractical for high-frequency strategies. This experience informed the subsequent decision to build a centralized matching engine with on-chain settlement.

2.3 Lessons Applied

Two design principles carried forward:

1. **Deterministic matching**: The matching engine must produce identical results for identical input sequences, regardless of execution environment.

2. **Audit trail**: Every state transition (order placement, fill, cancellation) must be recorded in an append-only log.

3 Centralized Exchange Architecture

3.1 System Overview

The CEX comprises five subsystems:

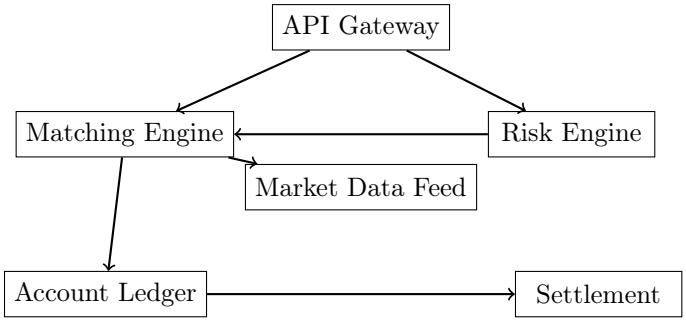

Figure 1: CEX subsystem topology.

3.2 Order Types

The engine supports:

- **Limit**: Execute at a specified price or better.
- **Market**: Execute immediately at the best available price.
- **Stop-Limit**: Place a limit order when a trigger price is reached.
- **IOC** (Immediate or Cancel): Fill what is available, cancel the remainder.
- **FOK** (Fill or Kill): Fill entirely or cancel entirely.

4 Order Matching Engine

4.1 Price-Time Priority

The matching engine implements strict price-time priority. For a given instrument, buy orders are sorted by descending price then ascending timestamp. Sell orders are sorted by ascending price then ascending timestamp.

Definition 4.1 (Order Priority)**.** *For orders o_1, o_2 on the same side:*

$$o_1 \succ o_2 \iff \begin{cases} p(o_1) > p(o_2) & \textit{(buy side)} \\ p(o_1) < p(o_2) & \textit{(sell side)} \\ t(o_1) < t(o_2) & \textit{if } p(o_1) = p(o_2) \end{cases} \tag{1}$$

where $p(o)$ is the limit price and $t(o)$ the arrival timestamp.

4.2 Matching Algorithm

The function $crosses(o_1, o_2)$ returns true when the incoming order's price would execute against the resting order: $p(buy) \geq p(sell)$.

4.3 Data Structures

Each price level maintains a doubly-linked list of orders. Price levels are stored in a red-black tree, providing $O(\log n)$ insertion, deletion, and best-price lookup.

Algorithm 1 Order matching with price-time priority.

```
function MATCH(order, book)
    remaining ← order.quantity
    opposite ← book.oppositeSide(order.side)
    while remaining > 0 and opposite ≠ ∅ do
        best ← opposite.peek()
        if ¬ crosses(order, best) then
            break
        end if
        fill ← min(remaining, best.quantity)
        price ← best.price                                  ▷ passive price
        EMITFILL(order, best, fill, price)
        remaining ← remaining − fill
        best.quantity ← best.quantity − fill
        if best.quantity = 0 then
            opposite.pop()
        end if
    end while
    if remaining > 0 and order.type = LIMIT then
        order.quantity ← remaining
        book.sameSide(order.side).insert(order)
    end if
end function
```

Table 1: Matching engine operation complexity.

Operation	**Complexity**
Place order (no match)	$O(\log n)$
Cancel order	$O(\log n)$
Match (per fill)	$O(1)$
Best bid/ask	$O(1)$

5 Market Data

5.1 WebSocket Feed

The market data feed distributes order book updates, trades, and ticker data over WebSocket connections. Three channels are provided:

- **Level 2**: Full order book depth, updated on every change.
- **Trades**: Individual fill events with price, quantity, and aggressor side.
- **Ticker**: 24-hour rolling statistics (open, high, low, close, volume).

5.2 Order Book Snapshots

Clients receive an initial snapshot followed by incremental deltas. The delta protocol ensures clients can reconstruct the full book state from any snapshot plus subsequent deltas:

$$B(t) = B(t_0) \oplus \Delta(t_0, t_1) \oplus \Delta(t_1, t_2) \oplus \cdots \oplus \Delta(t_{n-1}, t) \tag{2}$$

where $B(t)$ is the book state at time t and $\Delta(t_i, t_{i+1})$ is the delta between consecutive states.

6 Risk Management

6.1 Pre-Trade Risk Checks

Every incoming order passes through risk validation before reaching the matching engine:

1. **Balance check**: Sufficient free balance to cover the order (including fees).
2. **Position limit**: Aggregate position across open orders does not exceed per-account limits.
3. **Price collar**: Order price is within a configurable percentage of the reference price (last trade or mid-market).
4. **Rate limit**: Per-account order submission rate does not exceed the configured threshold.

6.2 Account Ledger

The account ledger maintains double-entry bookkeeping. Every balance change produces two entries: a debit and a credit. Holds are placed on balances when orders are submitted and released when orders are filled or canceled.

$$\text{available}(a) = \text{balance}(a) - \text{holds}(a) \tag{3}$$

7 Multi-Venue Aggregation

For instruments listed on multiple venues, the order router queries external order books and routes to the venue offering the best execution price, net of fees:

$$\text{effectivePrice}(v) = p(v) + f(v) \cdot p(v) \tag{4}$$

where $p(v)$ is the quoted price at venue v and $f(v)$ is the taker fee rate. The router selects $\arg\min_v \text{effectivePrice}(v)$ for buys and $\arg\max_v$ for sells.

8 Performance

Table 2: Matching engine throughput and latency.

Metric	**Value**
Orders matched per second	120,000
P50 matching latency	8 μs
P99 matching latency	45 μs
Instruments supported	200+
Concurrent WebSocket connections	50,000

9 Conclusion

The exchange architecture evolved from the on-chain Lightspeed DEX concept (2017) through a production centralized matching engine (2019). The CEX implementation provides price-time priority matching, pre-trade risk controls, double-entry accounting, and real-time market data distribution. The AMM research from Uniswap informed the decentralized trading component, while the centralized engine provided the throughput and latency characteristics required for

institutional participants. This architecture subsequently evolved into the Lux Exchange execution stack.

References

[1] H. Adams. Uniswap Whitepaper. `https://uniswap.org`, 2018.

[2] L. Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.

[3] T. H. Cormen et al. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[4] FIX Trading Community. FIX Protocol Specification. `https://fixtrading.org`, 2018.